

Codeforge Agents Test cases

1. Debug Broken Python

CODE:

```
def calculate_average(numbers): total = 0
for i in range(len(numbers)+1):
    total += numbers[i]

return total / len(number)
```

PROMPT:

Debug this code like a Senior Python Engineer.

1. Identify all bugs.
2. Explain root causes.
3. Estimate production impact.
4. Provide corrected code.
5. Suggest best practices.

2. Optimize Performance

CODE:

```
def find_duplicates(arr): duplicates = []
for i in range(len(arr)):
    for j in range(i+1, len(arr)):
        if arr[i] == arr[j]:
            duplicates.append(arr[i])

return duplicates
```

PROMPT:

Review this code as a Performance Optimization Agent.

1. Analyze time complexity.
2. Identify bottlenecks.
3. Optimize for 10 million records.
4. Compare before and after complexity.
5. Generate production-ready code.

3. Kubernetes YAML Review

CODE:

```
apiVersion: apps/v1 kind: Deployment metadata: name: payment-app
spec: replicas: 1
template: metadata: labels: app: payment-app
spec:
```

containers:

- name: payment-app
image: nginx:latest

PROMPT:

Act as a Kubernetes Platform Engineering Team.

1. Review this deployment.
2. Identify production risks.
3. Identify security vulnerabilities.
4. Suggest HA improvements.
5. Generate enterprise-ready YAML.

4. Java Security Review

CODE:

```
public class UserService {  
    public String login(String username,String password){  
  
        if(username.equals("admin") &&  
            password.equals("admin123"))  
            return "success";  
  
        return "failed";  
    }  
}
```

PROMPT:

Act as a Senior Java Security Architect.

1. Perform an OWASP review.
2. Identify vulnerabilities.
3. Explain attack scenarios.
4. Recommend enterprise fixes.
5. Generate secure production code.

5. Multi-Agent Code Review (Big WOW)

CODE:

```
def process_orders(orders):  
    revenue = 0  
  
    for order in orders:  
        if order["status"] == "completed":  
            revenue += order["amount"]  
  
    return revenue
```

PROMPT:

Act as a team of AI agents:

1. Reviewer Agent
2. Security Agent
3. Performance Agent
4. Architect Agent

Each agent must review independently.

Then generate:

- Findings
- Risks
- Recommendations
- Optimized code
- Final consolidated report

Demo Order

1. Debug Broken Python
2. Optimize Nested Loop
3. Kubernetes Review
4. Java Security Review
5. Multi-Agent Code Review

This sequence typically gets the strongest student reaction because they can clearly see the AI agents performing real software engineering tasks rather than just answering questions. 🚀

<https://cmrit-codeforge.streamlit.app/>

<https://cmrit-excel-insightforge-agent.streamlit.app/>

<https://console.groq.com/keys>

GROQ_API_KEY =

"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"

A2A (Agent-to-Agent) Protocol is an open standard that allows AI agents from **different frameworks, vendors**, or organizations to communicate, collaborate, and exchange tasks securely (HTTP, SMTP).

We started the week learning about agents. Today we saw how to deploy them using Docker, Kubernetes, and GitOps. The next evolution is agent teams collaborating through standards like MCP and A2A, eventually becoming digital workers that can operate business processes autonomously. The future belongs to systems that don't just answer—they act.

```
docker run -p 8501:8501 -e  
GROQ_API_KEY=gsk_FX7f7V6Bnmr5dzk6BOU2WGdyb3FYVPLfnTRwXBFmVhbMHnGm5  
NRi appars/excel-insightforge-agent:latest
```

codeforge-agents

1. Debug Broken Python

CODE:

```
def calculate_average(numbers): total = 0  
for i in range(len(numbers)+1):  
    total += numbers[i]  
  
return total / len(number)
```

PROMPT:

Debug this code like a Senior Python Engineer.

1. Identify all bugs.
2. Explain root causes.
3. Estimate production impact.
4. Provide corrected code.
5. Suggest best practices.

2. Optimize Performance

CODE:

```
def find_duplicates(arr): duplicates = []  
for i in range(len(arr)):  
    for j in range(i+1, len(arr)):
```

```
if arr[i] == arr[j]:
    duplicates.append(arr[i])
```

```
return duplicates
```

PROMPT:

Review this code as a Performance Optimization Agent.

1. Analyze time complexity.
2. Identify bottlenecks.
3. Optimize for 10 million records.
4. Compare before and after complexity.
5. Generate production-ready code.

3. Kubernetes YAML Review

CODE:

```
apiVersion: apps/v1 kind: Deployment metadata: name: payment-app
spec: replicas: 1
template: metadata: labels: app: payment-app
spec:
  containers:
  - name: payment-app
    image: nginx:latest
```

PROMPT:

Act as a Kubernetes Platform Engineering Team.

1. Review this deployment.
2. Identify production risks.
3. Identify security vulnerabilities.
4. Suggest HA improvements.
5. Generate enterprise-ready YAML.

4. Java Security Review

CODE:

```
public class UserService {
    public String login(String username,String password){

        if(username.equals("admin") &&
            password.equals("admin123"))
            return "success";

        return "failed";
    }
}
```

PROMPT:

Act as a Senior Java Security Architect.

1. Perform an OWASP review.
2. Identify vulnerabilities.
3. Explain attack scenarios.
4. Recommend enterprise fixes.
5. Generate secure production code.

5. Multi-Agent Code Review (Big WOW)

CODE:

```
def process_orders(orders):  
    revenue = 0  
  
    for order in orders:  
        if order["status"] == "completed":  
            revenue += order["amount"]
```

```
    return revenue
```

PROMPT:

Act as a team of AI agents:

1. Reviewer Agent
2. Security Agent
3. Performance Agent
4. Architect Agent

Each agent must review independently.

Then generate:

- Findings
- Risks
- Recommendations
- Optimized code
- Final consolidated report

Demo Order

1. Debug Broken Python
2. Optimize Nested Loop
3. Kubernetes Review
4. Java Security Review
5. Multi-Agent Code Review

This sequence typically gets the strongest student reaction because they can clearly see the AI agents performing real software engineering tasks rather than just answering questions. 🚀